

Robot Navigation with Reinforcement Learning

An interactive web app where a robot learns to navigate a grid world and reach a goal while avoiding obstacles, using tabular **Q-learning**. Built as a week-long project to learn the fundamentals of reinforcement learning applied to a simple robotics navigation problem.

Demo

Run locally (see below) or visit the deployed app: [\[add your Streamlit Cloud link here once deployed\]](#)

What it does

- A robot starts at the top-left corner of a 10x10 grid and must learn to reach a goal cell, avoiding user-placed obstacles.
- The robot learns purely through trial and error: it gets $+10$ for reaching the goal, -5 for hitting a wall/obstacle, and -1 per step (to encourage the shortest path).
- Over hundreds of training episodes, it builds a **Q-table** — a lookup table estimating "how good is each action from each grid position" — using the Q-learning update rule.
- The app visualizes the full learning process: the reward curve over training, a heatmap of the learned value of every cell, an arrow map of the learned policy, and a step-by-step playback of the trained robot's path.

Features

- **Interactive grid editor** — click any cell to toggle an obstacle on/off
- **Adjustable hyperparameters** — episodes, learning rate (alpha), discount factor (gamma), epsilon decay, all live in the sidebar
- **Learning curve** — raw + smoothed reward per episode, to visualize training progress
- **Q-value heatmap** — see what the robot learned about every cell, not just the final path
- **Policy arrows** — see the robot's learned "best action" from any point on the grid
- **Path playback** — scrub through the trained robot's route step by step

Tech stack

- Python
- [Streamlit](#) — web app framework
- NumPy — Q-table and grid math
- Matplotlib — all visualizations

How it works (technical summary)

The environment is a standard RL setup: `state` = robot's `(row, col)` position, `action` $\in$ {up, down, left, right}, and a `step(state, action)` function returning `(next_state, reward, done)`.

The agent uses **epsilon-greedy tabular Q-learning**:

- With probability epsilon, it takes a random action (exploration); otherwise it takes the currently best-known action (exploitation).
- Epsilon decays over training so the agent explores heavily early and exploits its knowledge later.
- After every step, the Q-table is updated via the standard temporal-difference rule:

$$Q(s,a) \leftarrow Q(s,a) + \alpha * (\text{reward} + \gamma * \max_{a'}(Q(s')) - Q(s,a))$$

Running locally

```
bash

git clone <your-repo-url>
cd <repo-folder>
pip install -r requirements.txt
streamlit run app.py
```

(On Windows, if `pip`/`streamlit` aren't recognized, use `python -m pip install -r requirements.txt` and `python -m streamlit run app.py` instead.)

Project structure

```
├─ app.py                # Streamlit web app
├─ requirements.txt       # Python dependencies
├─ robot_rl_day1.ipynb    # Prototyping notebooks (grid setup + rendering)
├─ robot_rl_day2.ipynb    # Movement / step function
├─ robot_rl_day3.ipynb    # Q-table + Q-learning update rule
├─ robot_rl_day4.ipynb    # Full training loop + learning curve
├─ robot_rl_day5.ipynb    # Q-value heatmap + policy visualization
└─ README.md
```

Possible extensions

- Swap tabular Q-learning for a small neural network (Deep Q-Network) to scale to larger/continuous state spaces
- Add moving obstacles for a harder navigation challenge
- Multi-agent version with several robots learning simultaneously

- Sim-to-real: deploy the learned policy on an actual small robot (e.g. a line-following bot with a grid-mapped environment)

Author

Built by [your name] as a robotics-focused machine learning project.